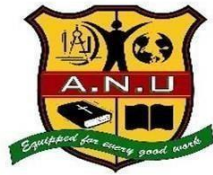


ALL NATIONS UNIVERSITY



INTERACTIVE QUIZ WEB APP WITH USER-GENERATED QUESTIONS AND
TIMER FUNCTIONALITY.

A report of the project submitted By

Sam Emmanuel Papa

To the

SCHOOL OF ENGINEERING

Submitted to fulfill the requirement for Diploma program

In

Computer Science

DEPARTMENT OF COMPUTER SCIENCE

April 2025

DECLARATION

I hereby declare that the project work entitled “INTERACTIVE QUIZ WEB APP WITH USER-GENERATED QUESTIONS AND TIMER FUNCTIONALITY” submitted in partial fulfillment of the requirements for the award of DIPLOMA IN COMPUTER SCIENCE & ENGINEERING, is a record of original work done by me under the supervision and guidance of Dr Theophilus Oware, Department of Computer Science, All Nations University Koforidua. This project work has not formed the basis for the award of any Diploma of similar title to any candidate of any university.

.....

Sam Emmanuel Papa

.....
Head of Department

.....
Supervisor

Table of Contents

Declaration	i
Acknowledgement	iii
Abstract	iv
1. Project Description	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Project Background	3
2. Web Usability	4
2.1 Web Usability	4
2.1.1 Common Definitions of Web Usability	4
2.1.2 Importance of Website Usability	5
2.1.3 Factors Influencing and Improving Web Usability	6
2.2 Website Performance	7
2.2.1 Measuring Web Performance	7
2.2.2 Factors Influencing Web Performance	8
2.2.3 14 Rules for Optimizing Performance	9
3. Requirement Analysis	15
3.1 Requirements Gathering	16
3.1.1 Literature Review and Guidelines	16
3.1.2 Observing Existing Quiz Platforms	16
3.1.3 Informal User Feedback	17
3.1.4 Iterative Prototyping	17
3.1.5 Technical Constraints	17
3.1.6 Functional Requirements	18
3.1.7 Non-Functional Requirements	18
3.2 Web Content Management System	18
4. Software Requirement Analysis	20
4.1 Functional Requirements	21
4.2 Non-Functional Requirements	23
4.3 Tools and Technologies Used	23
5. Project Implementation	24
5.1 Converting PSD Designs into HTML	24
5.1.1 Understanding PSD Design	24
5.1.2 Converting PSD to HTML Steps	24
6. Appendix	26
6.1 Sample Code (HTML, CSS, JavaScript)	26
7. References	49

ACKNOWLEDGEMENT

Jesus is the source of every blessing in my life. The inspiration for this work came entirely from Him. Glory and Honour be unto His Holy name for His guidance and provision.

I also owe a debt of gratitude to Dr Theophilus Oware, my project supervisor for his incredible support and prompt assistance in fulfilling my dream work and for his unusual encouragement which initiated an uncommon confidence in me that helped me to redefine my focus and priorities in my academic life.

I would like to thank all persons whose influence on my life contributed to the writing of this project. I am also grateful to the various authors whose works have been cited in this study. Special appreciation goes to:

FAMILY:

To my Father Mr Ebenezer Anane; whose effort has brought me far to this higher level of education. And also has been a source of inspiration to me and helped meet my financial challenges. I love you and the family, My late Mother Cythnia Abena Awukye, Grace Ama Nyarkoah, Alfred Kofi Anane.

Thanks to all for your immense love, unusual encouragement and support.

God bless you.

ABSTRACT

This project presents the design and development of an Interactive Quiz Web Application that integrates real-time quiz functionality with user participation through question submission. Built using HTML, CSS, and JavaScript, the application provides a responsive and engaging user interface that allows users to test their knowledge across various questions while being timed. A key feature of the system is its support for user-generated content, enabling users to submit their own multiple-choice questions via a dynamic form. These submissions are validated and stored locally using the browser's localStorage API, allowing for persistence across sessions without requiring a server or database.

The quiz system employs randomized question sequencing, real-time feedback on answer selection, and a countdown timer for each question to enhance engagement and simulate time pressured environments. At the end of each session, the user's score is calculated and displayed, offering a clear assessment of performance. The modular structure of the application allows for easy scalability and future enhancements such as category-based quizzes, server-side storage, or user authentication.

This project demonstrates core competencies in front-end web development, user interaction design, and local data management. It serves as a foundational tool for educational and recreational purposes and showcases the practical application of web technologies in creating dynamic and interactive learning platforms.

Introduction

In the digital era, web-based applications have become a central platform for learning, assessment, and knowledge sharing. Among these, quiz applications play a significant role in testing and reinforcing knowledge in an interactive and engaging manner. This project, titled "Interactive Quiz Web App with User-Generated Questions and Timer Functionality," focuses on the development of a browser-based quiz system designed to provide real-time feedback, track user performance, and encourage participation through question submission.

The application is developed using core front-end web technologies—HTML, CSS, and JavaScript—making it lightweight, easily deployable, and accessible across different devices without the need for server-side support. It features a timed quiz interface where users answer multiple-choice questions within a specified time limit per question. The inclusion of a countdown timer introduces an element of urgency and gamification, which enhances user focus and simulates exam-like conditions.

A standout feature of this application is its user-generated content functionality, which allows users to submit their own questions and multiple-choice answers through a structured form. These questions are validated and stored using the browser's `localStorage`, ensuring data persistence across sessions without a backend. This promotes interactivity and collaborative content creation, setting the application apart from static quiz tools.

Upon completing a quiz, users receive immediate feedback on their performance through a score summary. Questions are randomized to ensure a unique experience for each attempt, and answer feedback is color-coded for intuitive learning.

This project demonstrates practical knowledge in web development, user experience (UX) design, client-side data management, and basic interactivity principles. It serves as a flexible foundation that could be expanded in the future with server integration, user authentication, categorization, or analytics.

Motivation

The motivation behind developing this project stems from the increasing demand for interactive, accessible, and customizable learning tools in both academic and non-academic environments. Traditional assessment methods often lack flexibility and engagement, particularly in remote or self-paced learning contexts. This project seeks to bridge that gap by providing a lightweight, browser-based quiz application that users can easily interact with without needing to install software or connect to a server.

The inclusion of user-generated questions is particularly motivated by the desire to create a collaborative and dynamic learning environment. Allowing users to contribute their own questions promotes active learning, peer-to-peer knowledge sharing, and a sense of ownership over the educational process. This participatory model is more engaging than conventional static quizzes and encourages users to think critically when both answering and formulating questions.

The addition of a countdown timer further enhances the learning experience by introducing a level of challenge and urgency that simulates real-time exam conditions. This functionality not only boosts focus and time management skills but also prepares users for time-bound assessments in academic or professional settings.

From a developer's perspective, the project is a valuable opportunity to apply and demonstrate core front-end development skills, including DOM manipulation, event handling, data persistence via `localStorage`, and responsive UI design. The challenge of creating a fully functional application using only client-side technologies also encourages efficient coding practices and creative problem-solving.

Ultimately, this project is driven by a passion to make learning more interactive, inclusive, and efficient, using accessible web technologies to create meaningful user experiences.

Project Background

2.1 Web Usability

Web usability is a critical factor in the success of any web-based system. It refers to the ease with which users can interact with a website or web application to achieve their goals effectively and efficiently. For an educational tool such as an interactive quiz application, usability becomes even more important to ensure that learners of various levels can navigate, understand, and interact with the system with minimal effort or confusion.

A well-designed interface contributes to the overall learning experience by reducing cognitive load, improving task completion times, and increasing user satisfaction. This is particularly relevant in applications involving real-time interaction, such as quizzes with time constraints and dynamic question handling.

In the context of this project, web usability principles were carefully considered to ensure that users—regardless of their technical background—can easily start a quiz, answer questions, submit custom entries, and receive timely feedback. Layout clarity, button visibility, input validation, and responsive design all reflect usability concerns directly implemented in the system's front-end logic and styling.

2.1.1 The Common Definitions of Web Usability

Web usability has been defined and interpreted in various ways across academic and industrial contexts. Here are some widely accepted definitions:

- **Jakob Nielsen (1999)**, a pioneer in usability, defines web usability as *“a quality attribute that assesses how easy user interfaces are to use. The word ‘usability’ also refers to methods for improving ease of use during the design process.”* Nielsen breaks down usability into five components: learnability, efficiency, memorability, errors, and satisfaction—all of which are highly relevant to educational software like quiz applications.
- According to the **International Organization for Standardization (ISO 9241-11)**, usability is *“the extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use.”* This

definition emphasizes not only functionality but also the context and experience of the user, aligning with the project's aim to provide intuitive, fast, and satisfactory quiz interactions.

- **Shneiderman's Eight Golden Rules of Interface Design** also highlight aspects of usability such as consistency, user control, feedback, and error prevention. These principles directly apply to features implemented in the project code—such as real-time score feedback, error prevention in form submission, and a clearly visible user progress bar.
- In more practical terms, **web usability** is often seen as “*the ease with which users can learn, interact with, and remember how to use a website or application.*” For educational tools, this ensures minimal barriers between the user and learning objectives.

2.1.2 How and why website usability has been important?

Website usability is critical because it directly affects how users perceive and interact with digital platforms. In educational web applications like the one developed in this project, usability is not merely an enhancement—it is a necessity. A usable interface ensures that users can focus on learning rather than being distracted or hindered by a poorly designed system. In the context of this interactive quiz app, usability plays a vital role in several areas:

- **Ease of Navigation:** Users should be able to start, progress through, and finish quizzes without needing external instructions. The “Start,” “Next,” and “Restart” buttons are placed clearly and provide immediate feedback.
- **Efficiency and Speed:** The inclusion of a countdown timer ensures users remain engaged and focused, adding a gamified challenge while still maintaining simplicity.
- **Error Minimization:** Features such as disabling incorrect inputs, validating form entries, and highlighting the correct answers prevent frustration and guide users toward correct interaction patterns.
- **Inclusivity:** Allowing users to create and submit their own questions broadens engagement, making the app accessible and valuable for both quiz takers and content creators. Thus, usability is important not only for convenience but also for improving user performance, satisfaction, and learning outcomes.

2.1.3 The general factors influencing web usability and how to improve it

There are several general factors that influence web usability, all of which were considered in the design and development of the quiz application:

1. Clarity and Simplicity

A good web app should have a clear interface and intuitive layout. In this project, button labels like "Start Quiz", "Next", and "Submit Question" are straightforward and self-explanatory.

The layout avoids clutter by showing only necessary components—such as hiding the question form until the user explicitly requests it.

2. Responsiveness and Feedback

Immediate feedback is provided when a user selects an answer, helping them learn and stay engaged.

A countdown timer gives users a visual sense of urgency, while correct/incorrect answer indicators provide quick performance feedback.

3. User Control and Flexibility

Users can restart the quiz at any time or choose to submit new questions, giving them control over the flow of the application.

4. Accessibility and Error Prevention

The quiz form validates inputs before submission, ensuring that incomplete or improperly formatted questions are not accepted.

The use of localStorage ensures that user-generated content is preserved even if the page is refreshed, preventing data loss.

5. Aesthetic and Minimalist Design

The interface is kept clean with minimal distractions. Visual cues (such as color changes on correct/wrong answers) provide guidance without overwhelming the user.

How to Improve Web Usability Further

Based on the current project and general best practices, web usability can be improved further by:

- Adding accessibility features like screen reader support or keyboard navigation.
- Providing a visual progress bar in addition to numeric question indicators.
- Allowing users to review and change their answers before submitting the final score.
- Making the interface responsive across devices for better mobile usability.
- Offering customizable settings such as changing the time limit per question

Figure 2.1: Characteristics of a Usable Website

A usable website is defined by several core characteristics that ensure users can interact with it efficiently, effectively, and satisfactorily. The quiz web application developed in this project exemplifies these characteristics as outlined below:

Characteristic	Description	Implementation in Quiz Web App
Learnability	How easy it is for new users to accomplish basic tasks the first time they encounter the design.	The interface uses clear buttons like “Start Quiz”, “Next”, and “Restart Quiz”, making navigation intuitive even for first-time users.
Efficiency	How quickly users can perform tasks once they are familiar with the design.	The timed questions and streamlined question transitions help users move quickly through the quiz without unnecessary delays.
Memorability	How easily a returning user can remember how to use the system.	Familiar button styles, consistent layout, and intuitive feedback (e.g. color-coded answers) make the system memorable and easy to use again.

Error Tolerance	How well the system prevents errors and helps users recover from them.	The app disables answer buttons after a selection to prevent multiple submissions, and shows correct answers even if the user selects incorrectly.
Feedback	The extent to which the system informs users about what is going on.	Visual cues (correct/wrong colors), timers, score display, and progress indicators keep users informed about their actions and quiz status.
User Control	The degree of control users have over interactions.	Users can start, proceed to the next question, and restart the quiz at will. They can also contribute their own questions, enhancing interactivity.
Accessibility	The ability of the system to be used by people with varying abilities.	The form inputs, labels, and buttons are accessible and styled with good contrast. However, enhancements such as keyboard navigation and ARIA roles could further improve this aspect.

2.2 Website Performance

Website performance refers to how quickly and efficiently a website responds to user interactions and loads content. High-performing websites not only deliver a better user experience but also improve user retention, search engine rankings, and accessibility. In the context of a quiz web application, performance is critical because delays or lag in rendering questions, displaying timers, or processing answers can negatively impact usability and user satisfaction.

Good performance ensures that quiz transitions are smooth, scores are updated promptly, and feedback is given instantly. As modern users expect fast and responsive applications, performance optimization becomes a crucial aspect of web development.

Factors affecting website performance include page load time, script execution efficiency, image and asset optimization, server response time, and the effective use of browser storage mechanisms such as localStorage.

2.2.1 How to Measure Web Performance

Web performance can be measured using a combination of metrics, tools, and methodologies. These help developers identify bottlenecks and optimize the website for better speed and responsiveness. Common approaches include:

a) Performance Metrics

- **Page Load Time:** The time it takes for the page to fully load. A critical metric that reflects user perception of speed.
- **Time to Interactive (TTI):** Measures how long it takes for the page to become fully interactive.
- **First Contentful Paint (FCP):** The time it takes for the browser to render the first piece of content from the DOM.
- **Largest Contentful Paint (LCP):** Time taken to load the largest visible content element.
- **First Input Delay (FID):** Measures the time between a user's first interaction and the browser's response.

b) Tools for Measuring Web Performance

- **Google Lighthouse:** An automated tool built into Chrome DevTools. It audits performance, accessibility, and best practices.
- **Google PageSpeed Insights:** Analyzes a URL and provides detailed performance suggestions.
- **Chrome DevTools:** The Performance tab can be used to profile runtime performance, monitor resource loading, and debug JavaScript bottlenecks.

- **WebPageTest:** Allows developers to run performance tests from different locations and browsers.
 - **GTmetrix:** Combines performance scores and insights from both Lighthouse and PageSpeed.
- c) Real**

User Monitoring (RUM)

This method collects data from real users interacting with the site in production. It provides insights into actual performance across different devices, networks, and regions.

d) Synthetic Monitoring

Simulated tests are run under controlled conditions to analyze performance. Useful for benchmarking and regression testing.

e) Core Web Vitals

Introduced by Google, these metrics focus on the real-world user experience:

- **LCP** – loading performance
- **FID** – interactivity
- **CLS (Cumulative Layout Shift)** – visual stability

Relevance to the Project:

In the case of the quiz web app, performance measurement ensures that dynamic rendering of questions, timers, and form submissions are responsive. For instance, JavaScript execution (e.g., setting timers or updating the DOM) should be optimized to prevent lag, especially on lower-powered devices or slow networks

2.2.2 The General Factors Influencing Web Performance

Web performance is not determined by a single element, but by a combination of various technical and design choices that impact how quickly and efficiently a website operates. The performance of a web-based quiz application—such as the one developed in this

project—is influenced by several key factors. These include front-end code optimization, browser storage, DOM manipulation, external assets, responsiveness, and clientside execution. Below are the general factors relevant to this application:

a) JavaScript Execution Efficiency

The quiz app relies heavily on JavaScript to manage question rendering, user interaction, timer updates, and DOM manipulation. Poorly optimized scripts, such as unnecessary loops or excessive DOM access, can lead to increased CPU usage and delayed interactions. In the current code, functions like `setNextQuestion()`, `showQuestion()`, and `startTimer()` must be optimized for minimal execution delay.

b) DOM Manipulation

Frequent and inefficient manipulation of the Document Object Model (DOM) can degrade performance, especially when updating elements like the question text, answer buttons, and score. In this app, new buttons are created dynamically for each question, which requires efficient memory handling and proper element removal (`removeChild`) to prevent bloating the DOM tree.

c) Use of Timers

The use of `setInterval()` for countdown timers is essential to the quiz functionality. However, if not properly cleared (`clearInterval()`), timers can continue to run in the background and consume resources. In this code, timer cleanup is handled correctly in `resetState()` and `startTimer()`, which is crucial for maintaining performance across multiple questions.

d) Storage Mechanism (localStorage)

The app uses `localStorage` to save user-submitted questions. While this is a lightweight and efficient storage method for small datasets, frequent read/write operations can affect performance if not batched or minimized. Fortunately, in this case, data is only stored upon form submission, making the impact minimal.

e) CSS Rendering and Styling

Although CSS generally has low performance cost, complex or deeply nested styles can slow down rendering.

The app's CSS is lightweight and well-structured, but responsiveness and transitions (e.g., showing/hiding sections with `.hide` class) must be handled efficiently. Unused styles should also be removed to reduce CSS parsing time.

f) Page Structure and Layout

A clean and minimal HTML structure, as seen in the app, promotes faster rendering and easier maintenance. However, responsive layout handling through media queries and flexbox should be tested across devices to ensure no layout shifts occur that might trigger reflows and repaints, affecting perceived performance.

g) External Resources and Dependencies

Since the current app uses only local assets (CSS and JavaScript), it avoids external dependency delays. However, if libraries or fonts were added from third-party CDNs, DNS lookups and download latency could introduce load-time overhead.

h) Mobile Responsiveness

Modern users access websites from various screen sizes. The quiz app includes media queries to enhance responsiveness on wider screens. Ensuring that layout adapts gracefully to smaller screens (e.g., phones) is important to avoid slow interaction due to unoptimized rendering on mobile devices.

2.2.3 The Ideas Behind These 14 Rules

Improving website performance involves adhering to a set of best practices that reduce load time, optimize rendering, and enhance user experience. Steve Souders, a former web

performance expert at Yahoo! and Google, introduced 14 foundational rules that are still relevant today. These rules, when applied appropriately, improve both perceived and actual performance. Below is a breakdown of these rules and how they relate to the current quiz application.

Rule 1: Make Fewer HTTP Requests

- **Relevance to the Quiz App:** The app uses only one external stylesheet and one JavaScript file, minimizing HTTP requests. This is ideal, as fewer requests reduce the time it takes to load the page initially.

Rule 2: Use a Content Delivery Network (CDN)

- **Relevance:** This rule does not currently apply, since the project is entirely local. However, if images, fonts, or libraries were added, using a CDN would ensure faster access by serving content from geographically closer servers.

Rule 3: Add an Expires Header

- **Relevance:** This is a server-side rule. If deployed, the app could benefit from HTTP headers that cache static files (like CSS and JS) to reduce reload times.

Rule 4: Gzip Components

- **Relevance:** Also a server-side optimization. Gzipping CSS and JavaScript files on deployment compresses them, decreasing transfer time. Not applicable in local development but critical in production.

Rule 5: Put Stylesheets at the Top

- **Relevance:** The style3.css file is properly placed in the <head> section, ensuring styles are applied before rendering, avoiding flashes of unstyled content (FOUC).

Rule 6: Put Scripts at the Bottom

- **Relevance:** The script script3.js is loaded just before the closing </body> tag, which is a best practice to allow HTML to load before the JavaScript starts executing.

Rule 7: Avoid CSS Expressions

- **Relevance:** Not applicable. The app uses clean, static CSS with no dynamic CSS expressions (which were an old IE-only feature and are performance-heavy).

Rule 8: Make JavaScript and CSS External

- **Relevance:** The quiz app links its CSS and JavaScript externally, which improves caching and modular code organization. This follows the rule correctly.

Rule 9: Reduce DNS Lookups

- **Relevance:** Since no external domains (like fonts or CDNs) are used, DNS lookup delays are nonexistent in the current setup. If third-party resources are added, minimizing their domains would be beneficial.

Rule 10: Minify JavaScript and CSS

- **Relevance:** The current files are not minified, which is fine for development. However, minifying these files before deployment would reduce file sizes and improve load time.

Rule 11: Avoid Redirects

- **Relevance:** No redirects are used, which is good. Unnecessary redirects increase latency, especially on mobile networks. **Rule 12: Remove Duplicate Scripts**

- **Relevance:** The app only includes one instance of each script or stylesheet, which ensures that no redundant code is loaded. Duplicate script inclusion can increase load time and cause logic issues.

Rule 13: Configure ETags

- **Relevance:** Entity Tags (ETags) are server-related and used for cache validation. If the app is hosted, properly configuring ETags ensures efficient use of browser caching.

Rule 14: Make AJAX Cacheable

- **Relevance:** The current app does not use AJAX. If asynchronous data fetching (e.g., for loading questions from a server) is introduced in the future, it would be crucial to cache responses to reduce loading times.

Requirement Analysis

Requirement analysis is a critical stage in software development that involves understanding what the users need and what the system must do to fulfill those needs. This phase ensures that the end product meets its intended purpose and avoids costly revisions later. In the context of the quiz web application, requirement analysis includes identifying both functional and non-functional requirements based on stakeholder input, academic objectives, and usability considerations.

3.1 Requirements Gathering

Requirements gathering is the process of collecting the necessary information to define what the software system should do and how it should behave. This process involves engaging with stakeholders such as users, developers, and academic supervisors to understand the expectations and constraints of the system. The main goal is to produce a clear and comprehensive list of requirements that serve as a foundation for design and implementation.

For the quiz web application, requirements were gathered through the following approaches:

1. Literature Review and Academic Guidelines

As a final year project in computer science, the system was designed in alignment with academic standards and project scope expectations. The requirements were influenced by:

- Common patterns in quiz and assessment applications.
- Usability principles discussed in academic texts.
- Guidelines from the supervising instructor.

2. Observing Existing Quiz Platforms

To better understand user expectations and industry practices, existing quiz platforms (such as Kahoot, Quizizz, and Google Forms) were analyzed. This helped identify core features users expect, such as:

- Timer functionality.
- Score tracking.
- Immediate feedback.
- Simplicity and responsiveness.

3. Informal User Feedback

Discussions with peers and potential users helped gather preferences and pain points, such as:

- The desire for quick question interaction.
- Simple navigation and minimal page reloads.
- Visual clarity and fast response time.

4. Iterative Prototyping

An early version of the quiz app (as seen in the provided HTML, CSS, and JavaScript code) was used as a functional prototype. Feedback on this version helped refine the requirements based on:

- User confusion with the timer.
- Suggestions to allow users to submit their own questions.
- Requests for cleaner layout and spacing.

5. Technical Constraints

Technical constraints also shaped the requirements:

- The application is client-side only, using `localStorage` for persistence.
- No server-side backend, which limits user authentication and long-term data storage.
- Must be deployable with minimal infrastructure (e.g., can be hosted on GitHub Pages).

Preliminary Functional Requirements Identified

Based on the above sources, the following core functionalities were gathered:

- Start quiz button.
- Display of one question at a time with multiple-choice answers.
- Timer countdown with automatic question progression or quiz submission.
- Score calculation and display at the end.
- Option to add user-defined questions, stored in `localStorage`.

Preliminary Non-Functional Requirements

- **Usability:** The interface must be intuitive for users with minimal technical knowledge.
- **Performance:** Page must load quickly and handle logic instantly without lag.
- **Responsiveness:** The application should display well on both desktop and mobile browsers.
- **Maintainability:** Code should be modular and easy to extend.

3.2.1 Web Content Management System

A Web Content Management System (WCMS) is a software application that allows users to create, manage, and modify digital content without needing advanced technical skills. In this project, a traditional WCMS such as WordPress or Joomla was not used. Instead, a

custom solution was developed using HTML, CSS, and JavaScript. This approach is suitable for the scope and simplicity of a final year academic project.

The quiz web application enables users to add new quiz questions through an interface that captures inputs and stores them using the browser's `localStorage` feature. This allows users to manage quiz content without modifying the code directly, simulating a lightweight content management system. The saved questions are merged with the existing default questions, providing a dynamic and interactive experience.

This implementation avoids the complexity of a full WCMS by not requiring a server backend or database. The decision not to use a traditional CMS was based on several reasons: the application is entirely client-side, the content structure is simple, and the goal is to maintain a lightweight and fast-performing system.

There are several advantages to this custom approach. It simplifies deployment since the application can run on any static hosting service. It empowers users to modify quiz content directly through the interface. It enhances security by eliminating server-side vulnerabilities. Lastly, it ensures that the application is portable and functional entirely within the browser.

However, this method also has limitations. Since it uses `localStorage`, user-submitted questions are stored only on the device where they were created and are not shared across different users or devices. There is no user authentication or data synchronization. Additionally, as more content is added, managing it through the frontend may become difficult.

In summary, although the application does not use a traditional WCMS, it incorporates a basic content management mechanism suitable for its requirements. The use of JavaScript and `localStorage` effectively meets the goals of a dynamic, user-driven quiz system within the constraints of a student project.

Software Requirement Analysis

Web App Content Management System

To develop an effective and user-friendly interactive quiz application, it is essential to clearly define and gather both functional and non-functional requirements. The goal is to ensure the system meets user expectations and technical constraints. The requirements were gathered through observation of existing quiz platforms, informal user feedback, and a review of typical educational software functionalities.

1.2.1 Functional Requirements

These define the specific behaviours and functions the system must support. Based on the project scope and code implementation, the key functional requirements include:

Start and Navigate Through Quiz:

- The application must allow users to initiate a quiz through a "Start Quiz" button.
- Users should be able to proceed through questions using a "Next" button.

Randomized Questions:

- Questions should be randomly shuffled each time the quiz starts to provide variety.

Display of Questions and Answer Options:

- Each quiz question must be displayed with four answer options.
- Users must be able to select one of the options.

Answer Validation:

- The app must provide immediate feedback by highlighting correct and incorrect answers.

Score Calculation and Display:

- The application must calculate the user's score based on correct answers.
- Final scores should be displayed at the end of the quiz.

Timer Functionality:

- A countdown timer must be displayed during each question.
- If the timer runs out, the app should automatically disable answers and reveal the correct one.

Progress Indicator:

- The system must show the current question number out of the total.

Restart Quiz Feature:

- The quiz must offer an option to restart after completion.

User-Generated Question Submission:

- Users must be able to submit their own custom questions through a dedicated form.
- Submitted questions should include four options and a marked correct answer.

Persistence of Custom Questions:

- The system must save user-submitted questions locally using localStorage.

1.2.2 Non-Functional Requirements

- These define the quality and performance characteristics of the system.

Usability:

- The interface must be clean, responsive, and easy to navigate for users with minimal technical knowledge.

Responsiveness:

- The layout must adapt to different screen sizes including mobile, tablet, and desktop.

Performance:

- The app must load quickly and respond to user input without noticeable delay.

Data Persistence:

- User-submitted questions must be stored and persist even after the browser is refreshed.

Security (Basic):

- User input must be sanitized to prevent HTML/script injection or unintended data entry behavior.
- Scalability (Front-end Scope):
- The structure should support future features like categories, difficulty levels, or external data integration.

1.2.3 Tools and Technologies Used

- HTML5 & CSS3 – For structuring and styling the web interface.
- JavaScript (Vanilla) – For handling quiz logic, events, timer, and user input.
- Browser LocalStorage – For storing user-generated questions persistently on the client side.

Project Implementation

4.1 Convert PSD Designs into HTML Website Templates

The implementation phase of the quiz web application required translating the design mockups into functional, interactive HTML templates. These designs, often provided in PSD (Photoshop Document) format, were used as the foundation for creating a responsive and user-friendly web application.

Understanding the PSD Design

Before starting the conversion process, it is essential to understand the key components of the design, including layout, structure, colors, fonts, and interactive elements such as buttons, forms, and user interfaces. PSD files typically contain all the design elements such as headers, footers, navigation bars, content sections, and images in separate layers, making it easier to extract the required assets.

In the context of the quiz web application, the following design elements were critical to the conversion process:

1. Quiz Structure Layout: A clean and simple structure to house the question, answer buttons, timer, score, and progress elements.
2. Buttons and Controls: Consistent button styles for actions such as "Start Quiz," "Next Question," and "Submit."
3. User Interface for Content Submission: A form for users to submit their own quiz questions and answers, which would be visually distinct from the main quiz interface.
4. Responsive Design: Ensuring the layout adapts to different screen sizes for mobile, tablet, and desktop users.

Steps Involved in Converting PSD to HTML Template

1. Extracting Design Assets

The first step in converting a PSD file to an HTML template is extracting the necessary images, icons, and other visual elements. This includes buttons, background images, logos, and icons that will be used throughout the site. The assets should be saved in an appropriate format (e.g., PNG, JPEG) and optimized for web use to ensure the page loads quickly.

2. Setting up HTML Structure

The next step involves creating the basic HTML structure based on the layout provided in the PSD design. This includes setting up the necessary HTML tags for sections like headers, navigation, main content area, and footer. For the quiz web application, this includes elements such as:

- A container for the quiz.
- A timer section.
- A progress display.
- The question and answers display.
- Controls for navigation (Start, Next, Restart).

Appendix 1

Sample Code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-
scale=1">

  <title>Quiz App</title>

  <link rel="stylesheet" href="style3.css">

</head>

<body>

  <div class="container">

    <div id="quiz">

      <!-- Timer Display -->

      <div id="timer-container" class="hide">

        Time Left: <span id="timer">10</span>s

      </div>

      <div id="progress-container" class="hide">

        Question <span id="current-question">1</span> of <span
id="total-questions">10</span>

      </div>

      <div id="question-container" class="hide">

        <div id="question">Question text</div>

        <div id="answer-buttons" class="btn-container">

          <!-- Answer buttons will be dynamically added here -->
```

```

    </div>

</div>

    <div class="controls">

        <button id="start-btn" class="start-btn btn">Start
Quiz</button>

        <button id="next-btn" class="next-btn btn
hide">Next</button>

    </div>

    <div id="score-container" class="hide">

        <div id="final-score">Your score: <span
id="score">0</span></div>

        <button id="restart-btn" class="restart-btn btn">Restart
Quiz</button>

    </div>

</div>

<!-- Request to Submit Questions Button -->

<div id="request-submit-container">

    <button id="request-submit-btn" class="btn">Request to Submit
a Question</button>

</div>

    <div id="question-form-container" class="hide">

<h3>Submit Your Own Question</h3>

<form id="question-form">

    <div>

        <label for="user-question">Question</label>

        <input type="text" id="user-question" name="user-
question" required />

    </div>

```

```

<div>

    <label for="user-answer1">Answer 1</label>

    <input type="text" id="user-answer1" name="user-answer1"
required />

    <input type="radio" name="correct-answer" value="1"
required /> Correct

</div>

<div>

    <label for="user-answer2">Answer 2</label>

    <input type="text" id="user-answer2" name="user-answer2"
required />

    <input type="radio" name="correct-answer" value="2" />
Correct

</div>

<div>

    <label for="user-answer3">Answer 3</label>

    <input type="text" id="user-answer3" name="user-answer3"
required />

    <input type="radio" name="correct-answer" value="3" />
Correct

</div>

<div>

    <label for="user-answer4">Answer 4</label>

    <input type="text" id="user-answer4" name="user-answer4"
required />

    <input type="radio" name="correct-answer" value="4" />
Correct

</div>

<button type="submit" class="btn">Submit Question</button>

```

```

        </form>

    </div>

</div>

    <script src="script3.js"></script>
</body>
</html>

body {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    background: #f7f7f7;
    font-family: Arial, sans-serif;
    margin: 0;
}

.container {
    width: 90%;
    max-width: 600px;
    background: white;
    padding: 20px 30px;
    box-shadow: 0 0 15px rgba(0, 0, 0, 0.15);
    border-radius: 12px;
    text-align: center;
}

.btn {

```

```
margin: 8px;
padding: 12px 24px;
background: #007bff;
color: white;
border: none;
border-radius: 6px;
cursor: pointer;
transition: background 0.3s ease;
font-size: 16px;
}
```

```
.btn:hover {
  background: #0056b3;
}

.hide {
  display: none;
}

.controls {
  margin-top: 25px;
}

.btn-container {
  display: flex;
  flex-direction: column;
  align-items: stretch;
  margin-top: 15px;
```

```

}

.btn-container .btn {
  flex: 1;
}

.correct {
  background-color: #4caf50;
  color: white;
}

.wrong {
  background-color: #f44336;
  color: white;
}

#timer-container, #progress-container {
  font-size: 20px;
  margin-bottom: 15px;
}

#final-score {
  font-size: 24px;
  margin-bottom: 20px;
}

#question-form-container {
  margin-top: 40px;
  padding: 20px;
  background-color: #f9f9f9;
  border: 1px solid #ddd;
}

```

```

    border-radius: 8px;
}

#question-form div {
    margin-bottom: 15px;
}

#question-form input[type="text"] {
    width: 100%;
    padding: 10px;
    margin-top: 5px;
    border: 1px solid #ccc;
    border-radius: 4px;
}

#question-form label {
    display: block;
    font-size: 14px;
}

#question-form input[type="radio"] {
    margin-left: 10px;
}

@media (min-width: 600px) {
    .btn-container {
        flex-direction: row;
        flex-wrap: wrap;
        justify-content: center;
    }
}

```

```

    }

    .btn-container .btn {

        margin: 8px;

        width: 45%;

    }
}

#request-submit-container {

    margin-top: 20px;

}

.hide {

    display: none;

}

#question-form-container {

    margin-top: 40px;

    padding: 20px;

    background-color: #f9f9f9;

    border: 1px solid #ddd;

    border-radius: 8px;

}

#question-form div {

    margin-bottom: 15px;

}

#question-form input[type="text"] {

    width: 100%;

    padding: 10px;

```

```

    margin-top: 5px;

    border: 1px solid #ccc;

    border-radius: 4px;
}

#question-form label {

    display: block;

    font-size: 14px;
}

#question-form input[type="radio"] {

    margin-left: 10px;
}

const startButton = document.getElementById('start-btn');

const nextButton = document.getElementById('next-btn');

const restartButton = document.getElementById('restart-btn');

const questionContainer = document.getElementById('question-container');

const questionElement = document.getElementById('question');

const answerButtonsElement = document.getElementById('answer-buttons');

const scoreContainer = document.getElementById('score-container');

const finalScoreElement = document.getElementById('score');

const timerContainer = document.getElementById('timer-container');

const timerElement = document.getElementById('timer');

const progressContainer = document.getElementById('progress-container');

const currentQuestionElement = document.getElementById('current-question');

```

```

const totalQuestionsElement = document.getElementById('total-questions');

const questionForm = document.getElementById('question-form');

const userQuestion = document.getElementById('user-question');

const userAnswer1 = document.getElementById('user-answer1');

const userAnswer2 = document.getElementById('user-answer2');

const userAnswer3 = document.getElementById('user-answer3');

const userAnswer4 = document.getElementById('user-answer4');

const requestSubmitButton = document.getElementById('request-submit-btn');

const questionFormContainer = document.getElementById('question-form-container');

let shuffledQuestions, currentQuestionIndex;

let score = 0;

let timePerQuestion = 15; // Time in seconds for each question

let timer;

let storedQuestions = JSON.parse(localStorage.getItem('questions'))
|| [];

let defaultQuestions = [
  {
    question: 'What is the capital of France?',
    answers: [
      { text: 'Paris', correct: true },
      { text: 'London', correct: false },
      { text: 'Berlin', correct: false },
      { text: 'Madrid', correct: false }
    ]
  }
]

```

```

    },
    {
      question: 'Who is the CEO of Tesla?',
      answers: [
        { text: 'Elon Musk', correct: true },
        { text: 'Jeff Bezos', correct: false },
        { text: 'Bill Gates', correct: false },
        { text: 'Tony Stark', correct: false }
      ]
    }
  ];

let questions = defaultQuestions.concat(storedQuestions);

requestSubmitButton.addEventListener('click', () => {
  requestSubmitButton.classList.add('hide'); // Hide the request
  button

  questionFormContainer.classList.remove('hide'); // Show the form
});

questionForm.addEventListener('submit', (e) => {
  e.preventDefault();

  const questionText = userQuestion.value.trim();

  const answers = [
    { text: userAnswer1.value.trim(), correct:
document.querySelector('input[name="correct-
answer"]:checked').value === '1' },
    { text: userAnswer2.value.trim(), correct:
document.querySelector('input[name="correct-
answer"]:checked').value === '2' },
  ]

```

```

        { text: userAnswer3.value.trim(), correct:
document.querySelector('input[name="correct-
answer"]:checked').value === '3' },

        { text: userAnswer4.value.trim(), correct:
document.querySelector('input[name="correct-
answer"]:checked').value === '4' }

    ];

    const newQuestion = {

        question: questionText,

        answers: answers

    };

    questions.push(newQuestion);

    storedQuestions.push(newQuestion);

    localStorage.setItem('questions',
JSON.stringify(storedQuestions));

    questionForm.reset();

    alert('Question submitted successfully!');

});

startButton.addEventListener('click', startQuiz);

nextButton.addEventListener('click', () => {

    currentQuestionIndex++;

    setNextQuestion();

});

restartButton.addEventListener('click', () => {

    scoreContainer.classList.add('hide');

    startButton.classList.remove('hide');

    resetQuiz();

});

```

```

function startQuiz() {
  startButton.classList.add('hide');
  shuffledQuestions = questions.sort(() => Math.random() - 0.5);
  currentQuestionIndex = 0;
  score = 0;
  totalQuestionsElement.innerText = shuffledQuestions.length;
  questionContainer.classList.remove('hide');
  timerContainer.classList.remove('hide');
  progressContainer.classList.remove('hide');
  setNextQuestion();
}

function setNextQuestion() {
  resetState();
  showQuestion(shuffledQuestions[currentQuestionIndex]);
  updateProgress();
  startTimer();
}

function showQuestion(question) {
  questionElement.innerText = question.question;
  question.answers.forEach(answer => {
    const button = document.createElement('button');
    button.innerText = answer.text;
    button.classList.add('btn', 'answer-btn');
    if (answer.correct) {

```

```

        button.dataset.correct = answer.correct;
    }
    button.addEventListener('click', selectAnswer);
    answerButtonsElement.appendChild(button);
});
}

function resetState() {
    clearStatusClass(document.body);
    nextButton.classList.add('hide');
    clearInterval(timer);
    timerElement.innerText = timePerQuestion;
    while (answerButtonsElement.firstChild) {

answerButtonsElement.removeChild(answerButtonsElement.firstChild);
    }
}

function selectAnswer(e) {
    const selectedButton = e.target;
    const correct = selectedButton.dataset.correct === 'true';
    if (correct) {
        score++;
    }
    setStatusClass(selectedButton, correct);
    Array.from(answerButtonsElement.children).forEach(button => {
        button.disabled = true;
        setStatusClass(button, button.dataset.correct === 'true');
    });
}

```

```

    });

    clearInterval(timer);

    if (shuffledQuestions.length > currentQuestionIndex + 1) {
        nextButton.classList.remove('hide');
    } else {
        showScore();
    }
}

function startTimer() {
    let timeLeft = timePerQuestion;
    timerElement.innerText = timeLeft;

    timer = setInterval(() => {
        timeLeft--;
        timerElement.innerText = timeLeft;

        if (timeLeft <= 0) {
            clearInterval(timer);

            Array.from(answerButtonsElement.children).forEach(button => {
                button.disabled = true;

                if (button.dataset.correct === 'true') {
                    setStatusClass(button, true);
                }
            });

            if (shuffledQuestions.length > currentQuestionIndex + 1) {
                nextButton.classList.remove('hide');
            }
        }
    });
}

```

```

        } else {
            showScore();
        }
    }
}, 1000);
}

function updateProgress() {
    currentQuestionElement.innerText = currentQuestionIndex + 1;
}

function showScore() {
    questionContainer.classList.add('hide');
    timerContainer.classList.add('hide');
    progressContainer.classList.add('hide');
    scoreContainer.classList.remove('hide');

    finalScoreElement.innerText = `${score} /
    ${shuffledQuestions.length}`;
}

function resetQuiz() {
    resetState();

    scoreContainer.classList.add('hide');
    startButton.classList.remove('hide');
    questionContainer.classList.add('hide');
    timerContainer.classList.add('hide');
    progressContainer.classList.add('hide');
}

function setStatusClass(element, correct) {

```

```
clearStatusClass(element);  
if (correct) {  
    element.classList.add('correct');  
} else {  
    element.classList.add('wrong');  
}  
}
```

```
function clearStatusClass(element) {  
    element.classList.remove('correct');  
    element.classList.remove('wrong');  
}
```

Sample Project Mock-up

Time Left: 0s

Question 1 of 8

Which of the following has the fastest access time?

Cache
Memory

RAM

Hard Disk

Register

Next

Request to Submit a Question

References

- [1] T. Brinck, D. Gergle, and S. D. Wood, *Usability for the Web: Designing Web Sites that Work*. San Francisco, CA: Morgan Kaufmann, 2002.
- [2] J. Nielsen, "Jakob Nielsen's Website." [Online]. Available: <http://www.nngroup.com/>
- [3] H. Shahizan and L. Feng, "Evaluating the usability and content usefulness of web sites: A benchmarking approach," *Journal of Electronic Commerce in Organizations*, vol. 3, no. 2, pp. 46–56, Apr.–Jun. 2005.
- [4] K. Guenther, "Assessing web site usability," *Ebsco*, vol. 27, no. 2, pp. 65–70, 2003.
- [5] Human Factors International, "HFI brochure." [Online]. Available: http://www.humanfactors.com/downloads/documents/HFI_brochure.pdf, Oct. 2007.